ASSURE DEFI®

THE VERIFICATION GOLD STANDARD

SECURITY ASSESSMENT REPORT

VERIFIED BY ASSURE DEFI®

INTEGRITY • TRUST • CREDIBILITY

NAME:	IRONCLAD
STATUS:	FAIL
DATE:	18/04/2026

Risk Analysis

Vulnerability summary

Classification	Description
High	Vulnerabilities that can lead to **loss or theft of funds**, **permanent locking of assets**, **unauthorized minting/burning**, **protocol insolvency**, or **full control of the contract or protocol**. These issues are typically **exploitable on-chain** and require immediate remediation.
Medium	Vulnerabilities that **weaken protocol security or trust assumptions** but do not directly enable immediate loss of funds under normal conditions. Exploitation may require **specific conditions, privileged roles, or external failures**.
Low	Issues that have **limited direct impact** on funds or protocol integrity but may cause **unexpected behavior**, **reduced resilience**, or **future exploitability** if combined with other weaknesses.
Informational	Findings that **do not represent security vulnerabilities**, but highlight **code quality, clarity, maintainability, or best-practice improvements** that may enhance long-term safety and auditability.

SCOPE

Target Code And Revision

Project	Assure
Codebase	https://bscscan.com/address/0x4EE281947C63d97128a02C1bF7983c06EcEbB1BD
Audit Methodology	Static, Manual

Detailed Technical Report

HIGH

1. Unauthorized zero-allowance drain when combined fees reach 100%

Location: IRONCLADSECURITY.transferFrom()

Description:

transferFrom() computes tax and deflation from the requested amount, then applies _transfer(from, taxAddress, taxAmount) and _burn(from, deflationAmount) before calling super.transferFrom(from, to, amountToTransfer).

When taxBPS + deflationBPS == 10000, the net transfer amount becomes zero:

```
amountToTransfer = amount - taxAmount - deflationAmount = 0
```

In that case, the function still deducts value from from through tax and burn, but the final super.transferFrom(from, to, 0) consumes no allowance and succeeds with zero approval.

As a result, any arbitrary caller can trigger loss of funds from any token holder without being approved, provided the holder has sufficient balance and transfer

restrictions do not block the call.

This is a direct authorization failure in transferFrom, not merely a bad tokenomics setting.

Recommendation:

For the deployed contract:

immediately set taxBPS = 0

immediately set deflationBPS = 0

do not re-enable either on the live deployment

pause the token first if fees are currently active and pause is available

For future or replacement deployments:

spend allowance on the gross amount before any fee logic

add an invariant preventing taxBPS + deflationBPS >= 10000

2. Approved spender can exceed the granted allowance

Location: IRONCLADSECURITY.transferFrom()

Description:

The contract debits the holder's balance by the full requested amount, split across:

tax sent to taxAddress

deflationary burn

net transfer to recipient

However, allowance is only reduced by amountToTransfer, because the contract ultimately calls:

```
super.transferFrom(from, to, amountToTransfer);
```

This means a spender can repeatedly call transferFrom() and cause the token holder to lose more than the amount they approved.

Example:

holder approves spender for 100 tokens

tax is 10%, deflation is 0%

first call with amount = 100 removes 100 tokens from holder, but only 90 allowance is consumed

the spender can continue spending the residual allowance and extract more than 100 total from the holder

This breaks the expected ERC20 authorization invariant that a spender cannot cause more value to leave an account than was approved.

Recommendation:

For the deployed contract:

set taxBPS = 0

set deflationBPS = 0

keep both at zero permanently on this deployment

notify users and integrators that approvals were unsafe while fees were nonzero

For future or replacement deployments:

call _spendAllowance(from, msg.sender, amount) on the gross amount before any tax or burn logic

do not rely on super.transferFrom(..., amountToTransfer) for allowance accounting in fee-on-transfer logic

MEDIUM

1. ERC20 helper library does not verify false return values

Location: LibCommon.safeTransfer() and LibCommon.safeTransferFrom()

Description:

Both helper functions treat a low-level call as successful whenever the call itself does not revert. They do not check whether returned data decodes to false.

For ERC20 tokens that signal failure by returning false instead of reverting, these helpers may return success even though no transfer occurred.

This can cause accounting errors or settlement failures in any contract that relies on these helpers for token movement.

This is not currently the highest-risk issue in the provided token because the main token contract does not use these helpers for its core transfer flow, but it is still a genuine bug in the library.

Recommendation:

Replace these functions with OpenZeppelin SafeERC20, or explicitly validate return data:

if the call reverts, bubble the revert

if no return data is present, treat as success

if return data is present, decode it as bool and require it to be true

LOW

No low severity issues were found.

INFORMATIONAL

No informational severity issues were found.

Technical Findings Summary

Findings

Vulnerability Level	Total	Pending	Not Apply	Acknowledged	Partially Fixed	Fixed
🔴 High	2					
🟠 Medium	1					
🟡 Low						
🟢 Informational						

Assessment Results

Score Results

Review	Score
Global Score	**70/100**
Assure KYC	Not completed
Audit Score	70/100

The Following Score System Has been Added to this page to help understand the value of the audit, the maximum score is 100, however to attain that value the project must pass and provide all the data needed for the assessment. Our Passing Score has been changed to 84 Points for a higher standard, if a project does not attain 85% is an automatic failure. Read our notes and final assessment below. The Global Score is a combination of the evaluations obtained between having or not having KYC and the type of contract audited together with its manual audit.

Audit FAIL

Following our comprehensive security audit of the token contract for the **IronClad Security** project, the project did not fulfill the necessary criteria required to pass the security audit.

Disclaimer

Assure Defi has conducted an independent security assessment to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the reviewed code for the scope of this assessment. This report does not constitute agreement, acceptance, or advocating for the Project, and users relying on this report should not consider this as having any merit for financial Token in any shape, form, or nature. The contracts audited do not account for any economic developments that the Project in question may pursue, and the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude, and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are entirely free of exploits, bugs, vulnerabilities or deprecation of technologies. All information provided in this report does not constitute financial or investment in Token, nor should it be used to signal that any person reading this report should invest their funds without sufficient individual due diligence, regardless of the findings presented. Information is provided ‘as is, and Assure Defi is under no covenant to audit completeness, accuracy, or solidity of the contracts. In no event will Assure Defi or its partners, employees, agents, or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions or actions with regards to the information provided in this audit report. The assessment of Tokens provided by Assure Defi are subject to dependencies and are under continuing development. You agree that your access or use, including but not limited to any Tokens, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies with high levels of technical risk and uncertainty. The assessment reports could include false positives, negatives, and unpredictable results. The Token may access, and depend upon, multiple layers of third parties.